

Sistema de vision artificial para el control de un sumobot con ESP32-CAM

Juan Jose Prado Neira

Javier Rego Bargos

Junio de 2026

Resumen

Este documento describe el desarrollo final de un sistema de vision artificial para el control de un robot tipo sumobot. El prototipo utiliza una arquitectura distribuida: una ESP32-CAM captura y transmite video JPEG fragmentado por UDP, un portatil ejecuta el procesamiento de imagen y la toma de decision con Python y OpenCV, y un segundo controlador recibe ordenes UDP para accionar los motores. La solucion se organiza en cuatro bloques independientes: recepcion de video, procesamiento de imagen, decision y envio de respuesta. El sistema detecta lineas oscuras de carril, estima el centro navegable, identifica objetos que invaden la trayectoria, permite control manual y ejecuta una rutina automatica de expulsion basada en enfoque, aproximacion, empuje y reincorporacion. Las sesiones registradas suman 18.891 frames y 677,70 s de ejecucion, con una tasa media aproximada de 27,88 FPS. Los resultados muestran un prototipo funcional, interpretable y ajustable, adecuado para una demostracion completa de percepcion, decision y actuacion en robotica movil de bajo coste.

Palabras clave— vision artificial, OpenCV, ESP32-CAM, robotica movil, UDP, sumobot, control reactivo.

1. Introduccion

La robotica movil de bajo coste permite construir sistemas completos de percepcion y control sin recurrir a hardware especializado. En este proyecto se desarrolla un sumobot capaz de desplazarse sobre un carril definido por lineas oscuras, detectar obstaculos situados en su trayectoria y ejecutar maniobras para apartarlos. El interes del trabajo no reside solo en que el robot avance, sino en integrar una cadena completa: captura de imagen, transmision, procesamiento visual, decision, envio de comandos, control manual, depuracion y documentacion del codigo.

El planteamiento elegido evita entrenar un modelo de aprendizaje profundo y se basa en tecnicas clasicas de vision por computador. Esta decision reduce los requisitos de datos y computacion, facilita explicar cada paso del algoritmo y permite ajustar parametros durante las pruebas. La contrapartida es que el comportamiento depende de condiciones fisicas como iluminacion, contraste de las lineas, inclinacion de la camara y color de los objetos.

El sistema final se ejecuta en un portatil. La ESP32-CAM actua como sensor remoto y envia imagenes por UDP; el portatil procesa cada frame con OpenCV y decide una accion; el robot recibe comandos ASCII tambien

por UDP. Esta division descarga a las placas embarcadas de tareas pesadas y deja en el ordenador la parte mas flexible: visualizacion, ajuste de umbrales y registro de pruebas.

2. Objetivos

El objetivo general del proyecto es implementar un sistema funcional de vision y control para un sumobot usando componentes accesibles. A partir de este objetivo se establecen los siguientes objetivos especificos:

- Recibir video en tiempo real desde una ESP32-CAM mediante datagramas UDP y reconstruir frames utilizables por OpenCV.
- Detectar lineas de carril oscuras y estimar el centro navegable aun cuando solo una de las dos lineas sea visible.
- Detectar objetos oscuros dentro de la escena y evitar que sean confundidos con marcas del carril.
- Tomar decisiones automaticas de avance, curva, parada y maniobra de expulsion.
- Mantener un modo manual y una parada segura para poder probar el robot sin perder control sobre el movimiento.
- Generar herramientas de diagnostico visual y registros de depuracion que permitan justificar el funcionamiento.
- Documentar el codigo y la arquitectura para facilitar la entrega y el mantenimiento posterior.

3. Arquitectura general

La aplicacion se organiza como una cadena de cuatro bloques, coordinados desde `main.py`. Cada bloque reside en una carpeta propia dentro de `blocks/` e incluye un fichero `config.json` con sus parametros. El programa principal carga dinamicamente las clases de cada bloque, lee las configuraciones y ejecuta el bucle de control.

La comunicacion entre bloques se realiza mediante estructuras simples. El procesador de imagen devuelve un diccionario con la accion visual sugerida, confianza, offset, razon de deteccion, objetos localizados y varias imagenes de diagnostico. El bloque de decision recibe esa informacion y devuelve la accion final que debe enviarse



Figura 1: Vista de demostracion del sistema durante una ejecucion registrada.

Cuadro 1: Bloques principales del sistema.

Bloque	Responsabilidad
0_recibir_sumocam	Recepcion y reconstruccion UDP
1_procesar_imagen	Vision, carril, objetos y debug
2_toma_decision	Control automatico y manual
3_enviar_respuesta	Traduccion a comandos UDP

al robot. Esta interfaz reduce el acoplamiento entre partes y permite ajustar un bloque sin reescribir el resto.

La interfaz de ejecucion muestra una ventana de OpenCV con cuatro paneles: camara con superposicion de decision, mascara binaria, ROI detectada y vista bird/debug. Esta visualizacion fue esencial para ajustar umbrales, identificar falsos positivos y comprobar el estado del protocolo de expulsion de objetos.

4. Recepcion de video

El modulo de recepcion abre un socket UDP en 0.0.0.0:5005. La ESP32-CAM envia cada frame JPEG fragmentado en paquetes. Cada paquete incluye una cabecera con numero de frame, numero de paquete, total de paquetes, tamano del frame y tamano de payload. El receptor valida la cabecera, agrupa fragmentos por identificador de frame y descarta paquetes incompletos, caducados o incoherentes.

Cuando todos los fragmentos de un frame llegan correctamente, el JPEG se decodifica con `cv2.imdecode`. El bloque tambien contempla rotacion de imagen, configurada en este proyecto a 180 grados, y mantiene durante una pequena ventana temporal el ultimo frame valido. Esta ventana evita que la interfaz se congele ante perdidas puntuales de paquetes.

El uso de UDP se eligio por simplicidad y baja latencia. No garantiza entrega ordenada ni completa, pero en un sistema reactivo de video suele ser preferible descartar frames viejos antes que acumular retraso. El timeout de frame limita la memoria ocupada por fragmentos incompletos.

5. Procesado de imagen

El procesado visual se implementa en la clase `ImageProcessor`. Cada frame se redimensiona manteniendo relacion de aspecto y se procesa sobre una region de interes inferior, donde normalmente aparecen las lineas utiles para la navegacion inmediata. La configuracion final usa una anchura de 800 pixeles, `roi_top=0.6` y `roi_bottom=1.0`.

5.1. Mascara de carril

La deteccion de lineas combina dos criterios. El primero mide contraste local: se suaviza el canal de valor HSV con un desenfoque amplio y se resta el valor real para resaltar zonas oscuras sobre fondos claros. El segundo aplica un umbral absoluto sobre el valor del pixel. La mascara final es la union de ambos resultados, seguida de operaciones morfologicas de apertura, cierre y dilatado.

Este enfoque es robusto para lineas oscuras con iluminacion no uniforme. El contraste local permite conservar marcas que no siempre caen bajo el mismo umbral absoluto, mientras que el umbral absoluto ayuda cuando la linea es claramente negra.

5.2. Perspectiva y estimacion del carril

El sistema puede aplicar una transformacion de perspectiva configurable. En la configuracion actual se usa una transformacion normalizada que aproxima una vista cenital del area de interes. Si la transformacion produce una imagen invalida, el codigo vuelve automaticamente a la imagen original.

Sobre la mascara se extraen contornos. Para cada lado del frame se selecciona el mejor candidato de linea en funcion de posicion y area. Si aparecen dos lineas suficientemente separadas, el centro del carril se calcula como el punto medio entre ambas. Si solo aparece una linea, el sistema estima la otra usando un ancho de carril aprendido de detecciones anteriores o un valor inicial proporcional al ancho de la imagen. Para evitar cambios bruscos, el lado de la linea unica se estabiliza durante varios frames.

La salida incluye una confianza calculada a partir del area detectada y del centrado de la estimacion. El offset respecto al centro de la imagen se traduce en accion visual: avance, giro a izquierda, giro a derecha o parada por baja confianza. Ademas, se aplica suavizado para evitar alternancias rapidas entre acciones de giro.

5.3. Deteccion de objetos

El detector de objetos busca regiones oscuras de baja saturacion. Cada contorno se filtra por area minima, contacto con bordes, proporcion maxima de area de frame, relacion de aspecto y ratio de relleno. El objetivo es detectar objetos reales sin aceptar ruido, sombras o fragmentos de linea.

Cuando varios contornos aparecen apilados verticalmente, se fusionan si tienen solapamiento horizontal suficiente y una separacion compatible. Esta fusion evita que un unico obstaculo se interprete como varios objetos

independientes. Despues, la region del objeto se elimina de la mascara de lineas para reducir falsos positivos: un objeto oscuro no debe convertirse en una supuesta linea del carril.

6. Toma de decision

La decision final se implementa en `DecisionMaker`. El bloque conserva estado interno: movimiento habilitado, estado del regulador, informacion sobre lineas unicas, estado del protocolo de objetos y cooldown de la maniobra.

En seguimiento normal, si hay deteccion fiable de dos lineas, se calcula un error lateral y se transforma en una correccion mediante un regulador PID sencillo con ganancias configurables. Si el error cae dentro de una banda muerta, el robot avanza. Si el sistema ve una sola linea en la mitad inferior de la ROI, interpreta que puede estar entrando en una curva o esquina y mantiene una accion de curva tras un pequeno retardo.

El movimiento no se activa al arrancar. El usuario debe habilitarlo con `Enter` o espacio. Esta decision reduce riesgos durante pruebas: el sistema puede arrancar, recibir video y mostrar diagnostico sin mover el robot.

6.1. Protocolo de expulsion de objetos

Cuando el detector localiza un objeto con area suficiente, centrado razonable y posicion frontal, se activa un protocolo por estados:

1. **Enfoque:** el robot corrige a izquierda o derecha hasta centrar el objeto.
2. **Aproximacion:** avanza durante un numero fijo de frames.
3. **Empuje:** realiza una curva lateral para expulsar el objeto.
4. **Reincorporacion:** retrocede con curva para volver a orientarse respecto al carril.

Los tiempos se definen en frames, lo que simplifica la implementacion y evita depender de sensores adicionales. Al terminar, se aplica un cooldown para no reactivar inmediatamente el protocolo con el mismo objeto o con ruido residual.

7. Envio de comandos

El bloque `RobotCommandSender` traduce acciones semanticas a comandos ASCII para el firmware de motores. Las acciones directas incluyen avance, marcha atras, giro izquierdo, giro derecho y parada. Las curvas suaves se envian con comandos especificos como LL, RR, BL y BR, acompañados de velocidad nominal y delta de curva.

El emisor limita la frecuencia de envio con un intervalo minimo y evita repetir comandos iguales cuando no es necesario. La parada se fuerza siempre y, al cerrar el programa, se envian varias ordenes de S para dejar el robot en estado seguro. Tambien se incluye compatibilidad degradada para curvas hacia atras: si el robot responde con error, el sistema vuelve a una marcha atras simple.

8. Interfaz, control manual y registros

El bucle principal admite dos modos: automatico y manual. En modo automatico, las teclas `f`, `l`, `r` y `s` permiten forzar avance, izquierda, derecha o parada. En modo manual, `w`, `a`, `d` y `s` controlan directamente la accion mantenida. La tecla `m` conmuta entre modos y `q` o `Esc` cierran la aplicacion.

El sistema incluye un grabador de depuracion opcional. Cuando se activa, guarda videos de la vista principal, mascara, ROI y vista bird/debug. Tambien se genera un CSV por frame con tecla, modo, accion manual, accion enviada, confianza, offset, razon de vision y accion visual. Estos registros se usaron para analizar las sesiones finales.

9. Configuracion final

La configuracion se mantiene fuera del codigo para facilitar ajustes. La recepcion escucha en el puerto 5005 y aplica rotacion de 180 grados. El procesamiento trabaja a 800 pixeles de ancho, ROI inferior y deteccion de objetos activada. El bloque de decision arranca con movimiento deshabilitado y utiliza umbrales de confianza, banda muerta y ganancias PID moderadas. El envio apunta a la IP del robot configurada y limita las velocidades para pruebas seguras.

Cuadro 2: Parametros representativos de la configuracion final.

Parametro	Valor	Bloque
Puerto video UDP	5005	Recepcion
Rotacion	180 grados	Recepcion
Anchura de proceso	800 px	Vision
ROI vertical	0,6–1,0	Vision
Confianza minima	0,2	Decision
Banda muerta	15 px	Decision
Velocidad avance	20 %	Envio
Velocidad giro	30 %	Envio

10. Validacion experimental

La validacion se realizo mediante ejecuciones registradas con el sistema de debug. Los datos no deben interpretarse como un benchmark universal, ya que dependen del entorno fisico de prueba, pero si permiten comprobar estabilidad, frecuencia de procesamiento y distribucion de detecciones.

En el total de frames, el sistema detecto dos lineas en 12.340 casos (65,3 %), una sola linea izquierda en 2.907 casos (15,4 %), una sola linea derecha en 1.416 casos (7,5 %) y ausencia de lineas en 2.159 casos (11,4 %). La razon de baja confianza aparecio en 69 frames (0,4 %). La confianza media global fue 0,744 y el offset absoluto medio registrado fue de 48,7 pixeles.

Las acciones enviadas mas frecuentes fueron avance (4.395 frames), curva izquierda (2.452), marcha atras cur-

Cuadro 3: Resumen de ejecuciones registradas.

Sesion	Frames	Tiempo (s)	FPS	Conf.
13:04	1.518	54,74	27,73	0,579
13:04b	2.407	87,06	27,65	0,526
13:06	2.993	107,50	27,84	0,765
13:08	11.973	428,40	27,95	0,803
Total	18.891	677,70	27,88	0,744

va izquierda (1.240), curva derecha (767) y parada (10). En 10.027 frames no se envió una acción nueva, principalmente por movimiento deshabilitado, repetición de comando o limitación temporal del emisor. Este comportamiento es intencionado: evita saturar el canal UDP y reduce cambios innecesarios en los motores.

11. Discusion

El sistema cumple el objetivo de integrar percepción, decisión y actuación en tiempo real. Una frecuencia cercana a 28 FPS es suficiente para un sumobot de laboratorio, especialmente al trabajar con velocidades moderadas. La estructura por bloques favorece el mantenimiento: recepción, visión, decisión y envío se pueden probar o ajustar de forma independiente.

La principal fortaleza del enfoque es la interpretabilidad. Cada decisión puede relacionarse con una máscara, un contorno, un offset y una razón de detección. Esto facilita depurar el sistema durante una demostración. También permite explicar el proyecto sin recurrir a modelos opacos ni a procesos de entrenamiento externos.

Las limitaciones son coherentes con una solución basada en umbrales. Cambios fuertes de iluminación, sombras, reflejos o líneas con poco contraste pueden reducir la confianza. La detección de objetos asume que el obstáculo es oscuro y de baja saturación, por lo que objetos claros o muy reflectantes podrían no activarla. Además, el protocolo de expulsión usa duraciones fijas en frames: funciona para el montaje probado, pero no mide distancia física ni orientación real del robot.

Otra limitación importante es la ausencia de odometría. El sistema decide a partir de imagen y tiempo, pero no conoce desplazamiento real, velocidad de las ruedas ni ángulo del chasis. Incorporar sensores adicionales permitiría cerrar el lazo de control con mayor precisión.

12. Conclusiones

Se ha conseguido implementar un prototipo funcional capaz de recorrer un camino delimitado con cinta aislante negra, detectar objetos situados dentro de la vía y ejecutar una secuencia automática para retirarlos. El comportamiento final no se limita al seguimiento de carril: cuando aparece un obstáculo, el robot lo centra, avanza hacia él, realiza una maniobra de empuje para sacarlo de la zona de paso y posteriormente da marcha atrás con curva para reincorporarse al circuito. Esta secuencia puede repetirse sucesivamente, de modo que el robot

continúa recorriendo la pista y atendiendo nuevos objetos detectados.

Las pruebas prácticas muestran que la detección y la expulsión funcionan de forma satisfactoria con objetos cotidianos como una figura o un cargador. En estos casos el sistema los localiza correctamente, activa el protocolo de expulsión y logra desplazarlos fuera del camino sin perder de forma habitual la referencia de la vía. La interfaz de depuración también permite observar que las decisiones tomadas se corresponden con la información visual disponible: líneas detectadas, offset respecto al centro y presencia del objeto.

La principal limitación observada no procede solo del algoritmo, sino de la interacción física entre el robot y el obstáculo. Según la forma y el material del objeto, este puede resbalar sobre la cuna o cuna frontal del robot y salirse antes de lo previsto. En algunas situaciones, si el objeto queda parcialmente fuera de la vía pero todavía aparece en la imagen, el sistema puede volver a detectarlo y continuar la maniobra, lo que podría hacer que el robot se abra más de lo deseado. No obstante, durante las pruebas este caso no fue el comportamiento habitual y el sistema resultó estable con los objetos usados como referencia.

Como trabajo futuro se propone mejorar la parte mecánica de contacto con el objeto para reducir deslizamientos, incorporar odometría o una IMU para conocer mejor la posición real del robot, medir latencia extremo a extremo y adaptar la duración del protocolo de expulsión a una estimación de distancia en lugar de usar un número fijo de frames. También podría ampliarse la detección de objetos para cubrir colores y materiales menos favorables que los probados.

Referencias

- [1] G. Bradski, “The OpenCV Library,” *Dr. Dobb’s Journal of Software Tools*, 2000.
- [2] Espressif Systems, “ESP32 Series Datasheet,” 2024.
- [3] R. Szeliski, *Computer Vision: Algorithms and Applications*. Springer, 2022.
- [4] Sphinx documentation team, “Sphinx documentation generator,” 2026.